Modeling Anthropogenic CO2 Accumulation: A Bayesian Comparison of Linear and Accelerating Trends

Álvaro Méndez Rodríguez de Tembleque[a]

[a] *Universita degli Studi di Padova, Department of physics and astronomy "Galileo Galilei", City, 12345*

Abstract

The Mauna Loa Observatory is a premier baseline site for monitoring historical and current atmospheric carbon dioxide (CO2) concentrations. This report investigates whether the long-term increase in atmospheric CO2 is best described by a constant linear increase or an accelerating quadratic trend. Using monthly average CO2 data from the NOAA Global Monitoring Laboratory, we address missing values and compute annual averages to isolate the long-term trend from seasonal variations. We perform a Bayesian analysis to estimate the parameters of both a linear model and a quadratic model, with a specific focus on the acceleration parameter. The two models are then evaluated and compared using an approximate Bayes factor to determine which model better captures the underlying trend dynamics. Finally, utilizing the best-fitting model, we project the annual atmospheric CO2 concentrations for the next decade, providing predictive estimates complete with Bayesian credible bands to quantify uncertainty.

1. Introduction

1.1. Scientific Context

1.2. Objectives

2. Data Description and Preprocessing

The dataset used in this analysis consists of monthly average atmospheric CO2 concentrations measured at the Mauna Loa Observatory, Hawaii, from 1958 to 2026. The data is sourced from the NOAA Global Monitoring Laboratory and is available in a TXT format. Each record includes the date of measurement and the corresponding CO2 concentration in parts per million (ppm).

The datasset is structured as follows:

- `year`: The year of the measurement.
- `month`: The month of the measurement.
- `decimal_date`: The decimal representation of the date (e.g., 2020.5 for June 2020).

*Corresponding author

Email address: `alvaro.mendezrodriguezdetembleque@studenti.unipd.it` (Álvaro Méndez Rodríguez de Tembleque)

- `monthly_average`: The average CO2 concentration for that month in ppm.
- `deseasonalized`: The monthly average CO2 concentration with the seasonal component removed.
- `num_days`: The number of days in the month used for averaging.
- `st_dev`: The standard deviation of the daily CO2 measurements for that month.
- `unc_mon_mean`: The uncertainty in the monthly mean CO2 concentration.

2.1. Dataset Overview

See code 1 in Appendix for the implementation.

Table 1: Summary statistics of the Mauna Loa CO2 dataset

Variable	Missing	Mean	SD	Min	Median	Max
year	0	1991.75	19.69	1958.00	1992.00	2026.00
monthly_average	0	361.02	33.19	312.42	356.26	431.12
deseasonalized	0	361.02	33.13	314.44	356.42	428.72
num_days	195	25.48	4.11	2.00	26.00	31.00
st_dev	196	0.51	0.20	0.15	0.48	1.31
unc_mon_mean	194	0.19	0.08	0.00	0.18	0.58

As shown in Table 1, the dataset contains some missing values, we will clean the dataset so that we can perform our analysis on a complete dataset.

2.2. Data Cleaning

The dataset contains some missing values, which are we droped all the rows with missing values limiting our analysisto the period from 1974 to 2026.

See code 2 in Appendix for the implementation.

Table 2: Summary statistics of the Mauna Loa CO2 dataset after removing missing values

Variable	Missing	Mean	SD	Min	Median	Max
year	0	1999.90	15.01	1974.00	2000.00	2026.00
monthly_average	0	373.36	28.38	327.28	369.80	431.12
deseasonalized	0	373.36	28.30	329.79	369.40	428.72
num_days	0	25.52	4.00	7.00	26.00	31.00
st_dev	0	0.51	0.20	0.15	0.48	1.31
unc_mon_mean	0	0.20	0.08	0.06	0.18	0.58

We see that the dataset is now complete, with no missing values, and the summary statistics are consistent with the original dataset, indicating that the data cleaning process has not introduced any significant bias. Also the missing values were mostly concentrated in the early years of the dataset, and did not affect the overall trend of increasing CO2 concentrations over time.

With the cleaned data, we can get an overall view of the data by plotting the distributions of the different variables.

See code 3 in Appendix for the implementation.

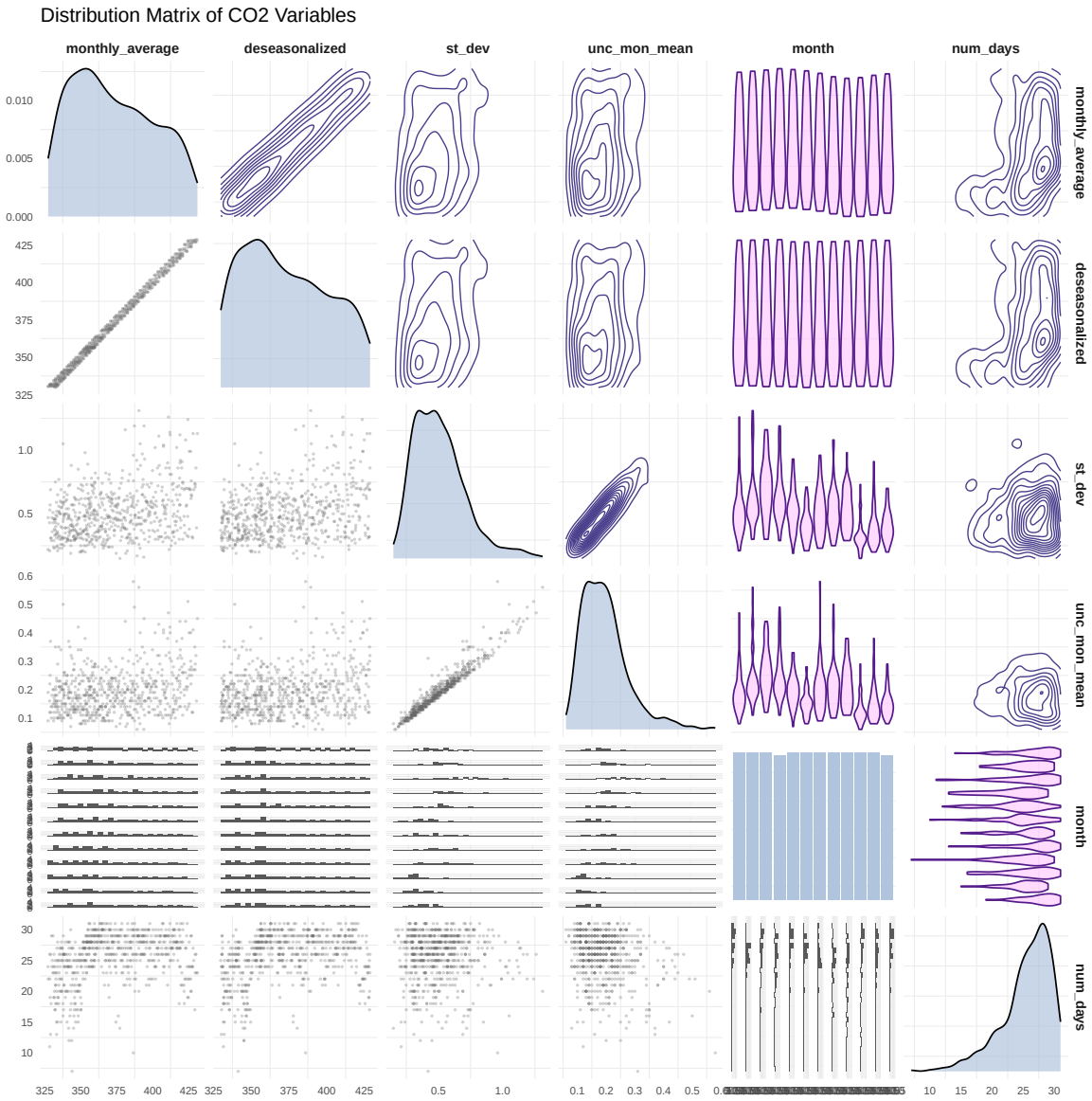

Figure 1: Distribution and correlation of the variables in the dataset. The diagonal panels show the distribution of each variable, while the upper panels show the correlation coefficients between pairs of variables. The lower panels display scatter plots

2.3. Aggregation

While the original dataset is what the analysis will focus on, there might be useful to compute other quantities in order to understand better the data, those quantities are:

- `yearly_mean`: The average CO2 concentration for each year, computed by averaging the monthly averages for that year (See Figure 3).
- `day_mean`: The average CO2 concentration for each number of days measured in a month, computed by averaging the monthly averages for each unique value of `num_days` (See Figure 4).
- `seasonal_deviation`: The deviation of the monthly average CO2 concentration from the deseasonalized value, computed as `monthly_average - deseasonalized` (See Figure 5).

2.4. Exploratory Visualization

In this section, we will explore the data through various visualizations to understand the underlying patterns and relationships between the variables.

2.4.1. Trend Analysis

See code 4 in Appendix for the implementation.

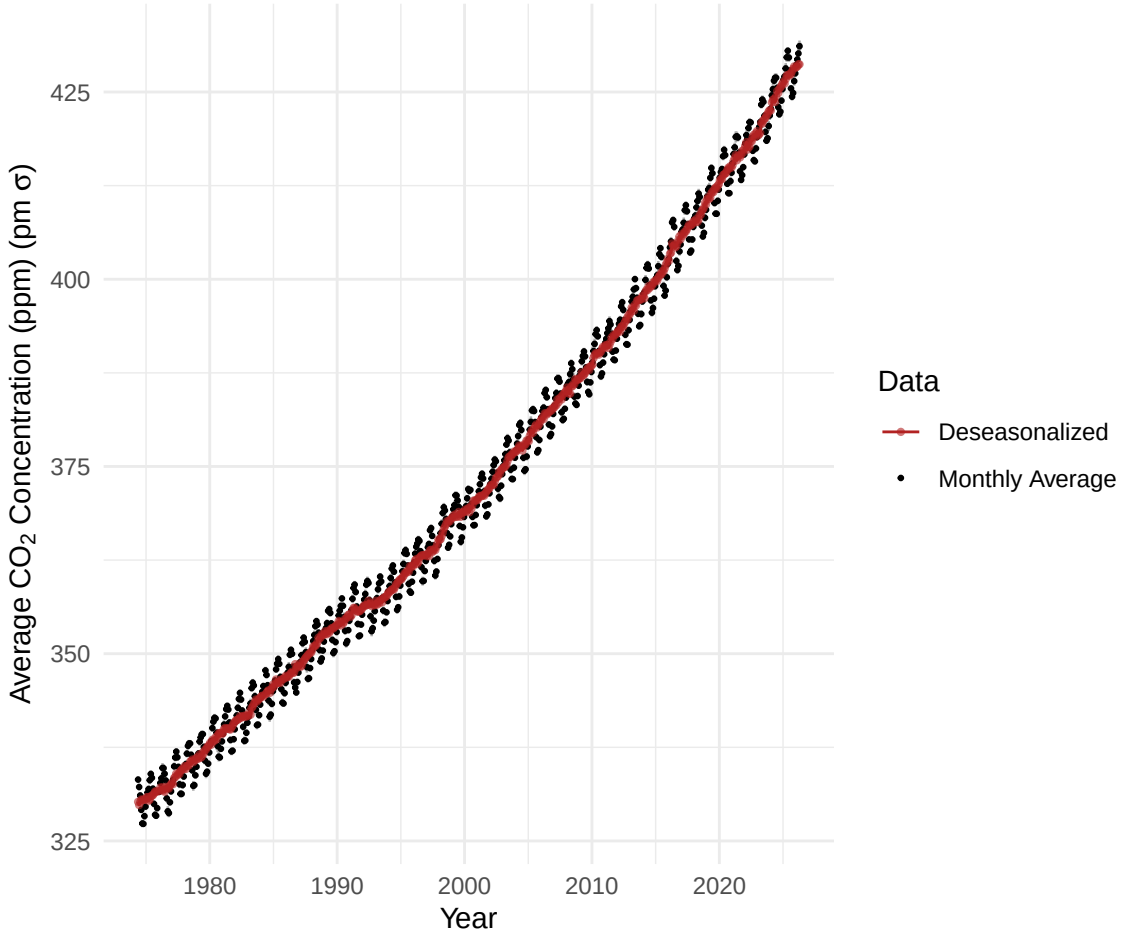

Figure 2: Monthly average CO2 concentrations. The plot shows a clear upward trend in atmospheric CO2 levels over time.

From Figure 2, we observe a clear upward trend in atmospheric CO2 levels over time, with the data showing a consistent increase in annual average CO2 concentrations. This visual evidence suggests that the long-term trend may not be purely linear, prompting us to consider both linear and accelerating models for further analysis.

See code 5 in Appendix for the implementation.

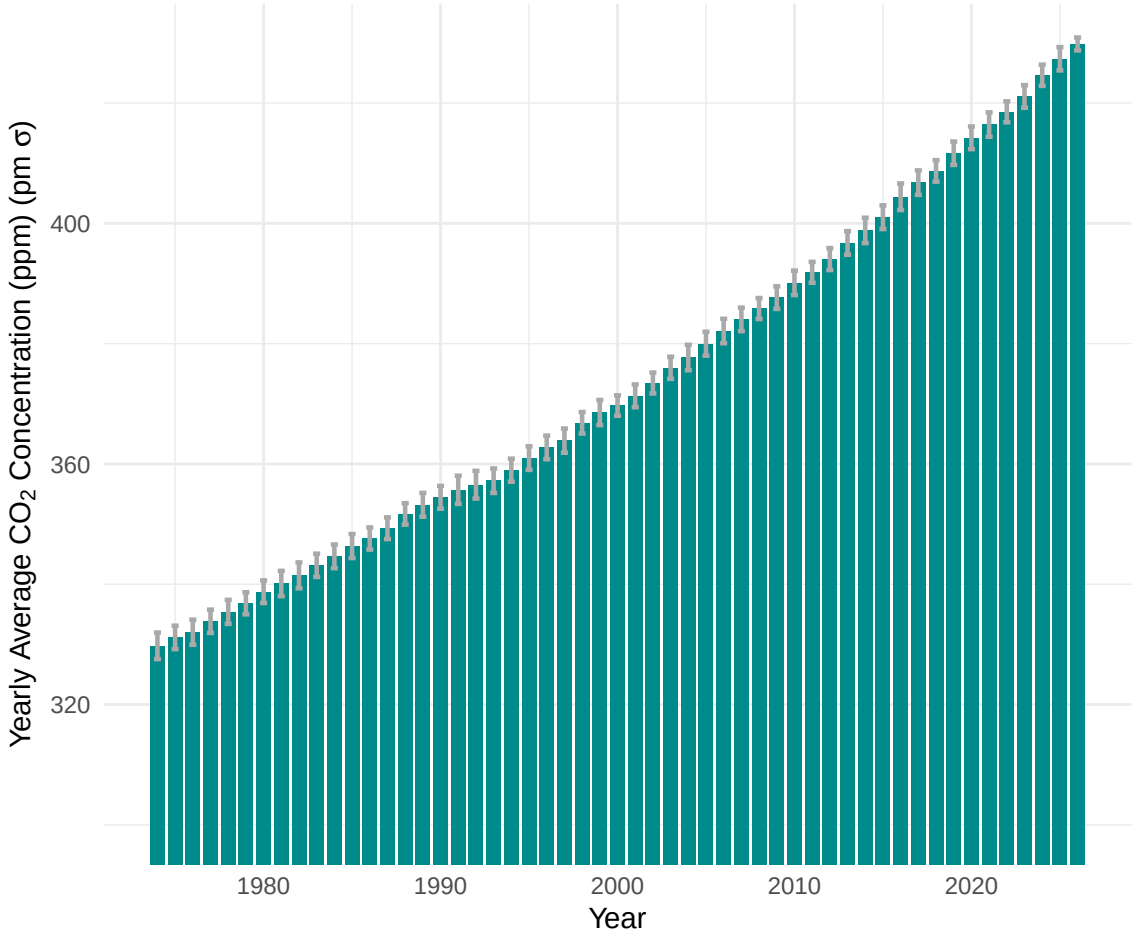

Figure 3: Yearly average CO2 concentrations. The plot shows a clear upward trend in atmospheric CO2 levels over time.

In Figure 3, we can see the yearly average CO2 concentrations, which also show a clear upward trend over time, reinforcing the observation from the monthly data. We can also examine weather there is a relationship between the number of days measured in a month and the monthly average CO2 concentration,

See code 6 in Appendix for the implementation.

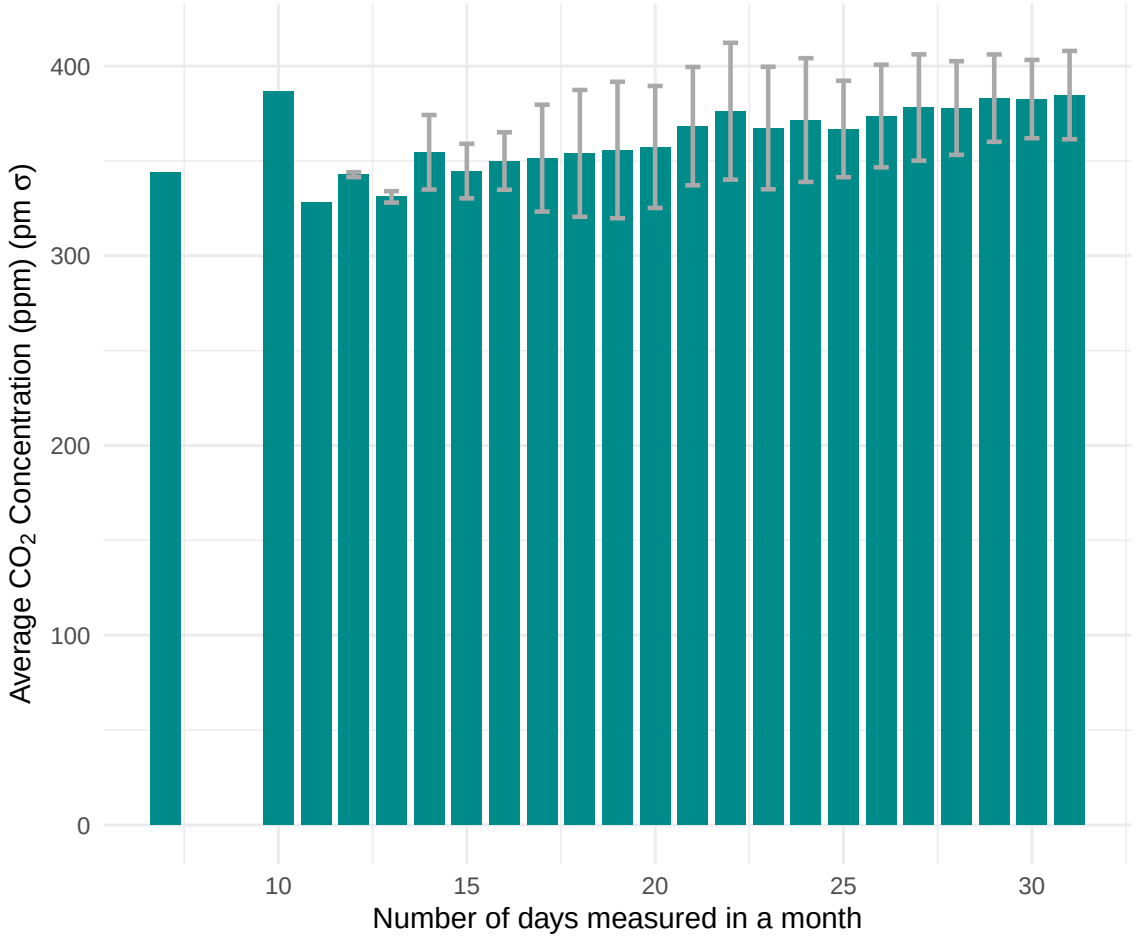

Figure 4: Daily average CO2 concentrations. The plot shows a clear upward trend in atmospheric CO2 levels over time.

which would mean that the more days measured in a month, the higher the monthly average CO2 concentration, this could be due to the fact that months with more measurements are more likely to capture higher CO2 levels, especially during periods of rapid increase. In Figure 4, we observe that the monthly average CO2 concentration is uniformly distributed across different values of `num_days`, suggesting that the number of days measured in a month does not have a significant impact on the monthly average CO2 concentration.

2.4.2. Seasonal Deviation Analysis

In this analysis the data can be understanded as a time series with a seasonal component, some months of the year have higher CO2 concentrations than others, this is due to the fact that during the summer months, plants are actively photosynthesizing and absorbing CO2 from the atmosphere, and people are more likely to be outside and emitting CO2, while during the winter months, plants are dormant and less CO2 is absorbed, and people are more likely to be inside and emitting less

CO2.

See code 7 in Appendix for the implementation.

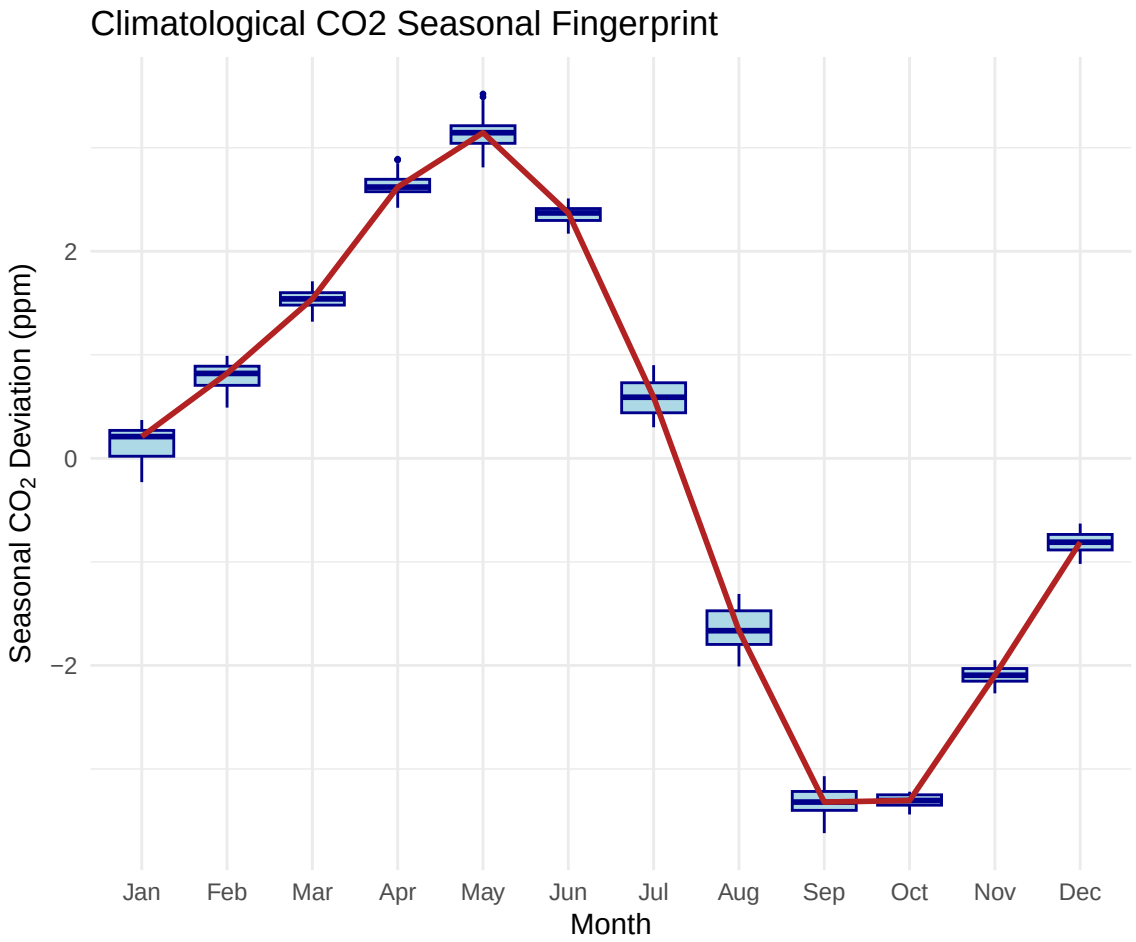

Figure 5: Box plot of seasonal deviations in CO2 concentrations.

2.4.3. Uncertainty Analysis

See code 8 in Appendix for the implementation.

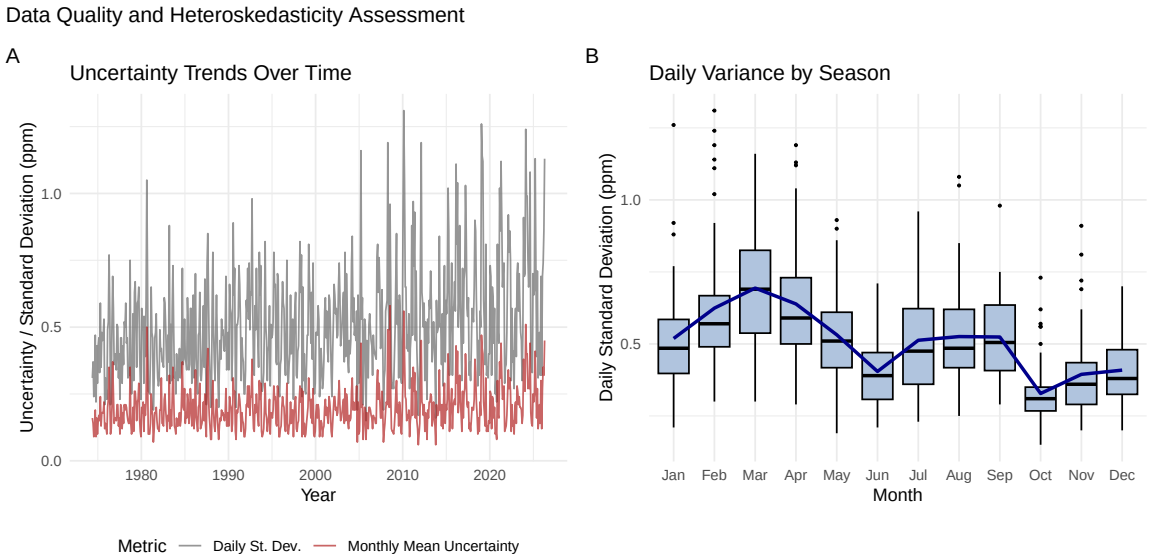

Figure 6: Uncertainty analysis of the CO2 measurements. The left panel shows the trends of daily standard deviation and monthly mean uncertainty over time, while the right panel shows the distribution of daily standard deviation across different months.

2.4.4. Correlation Analysis

See code 9 in Appendix for the implementation.

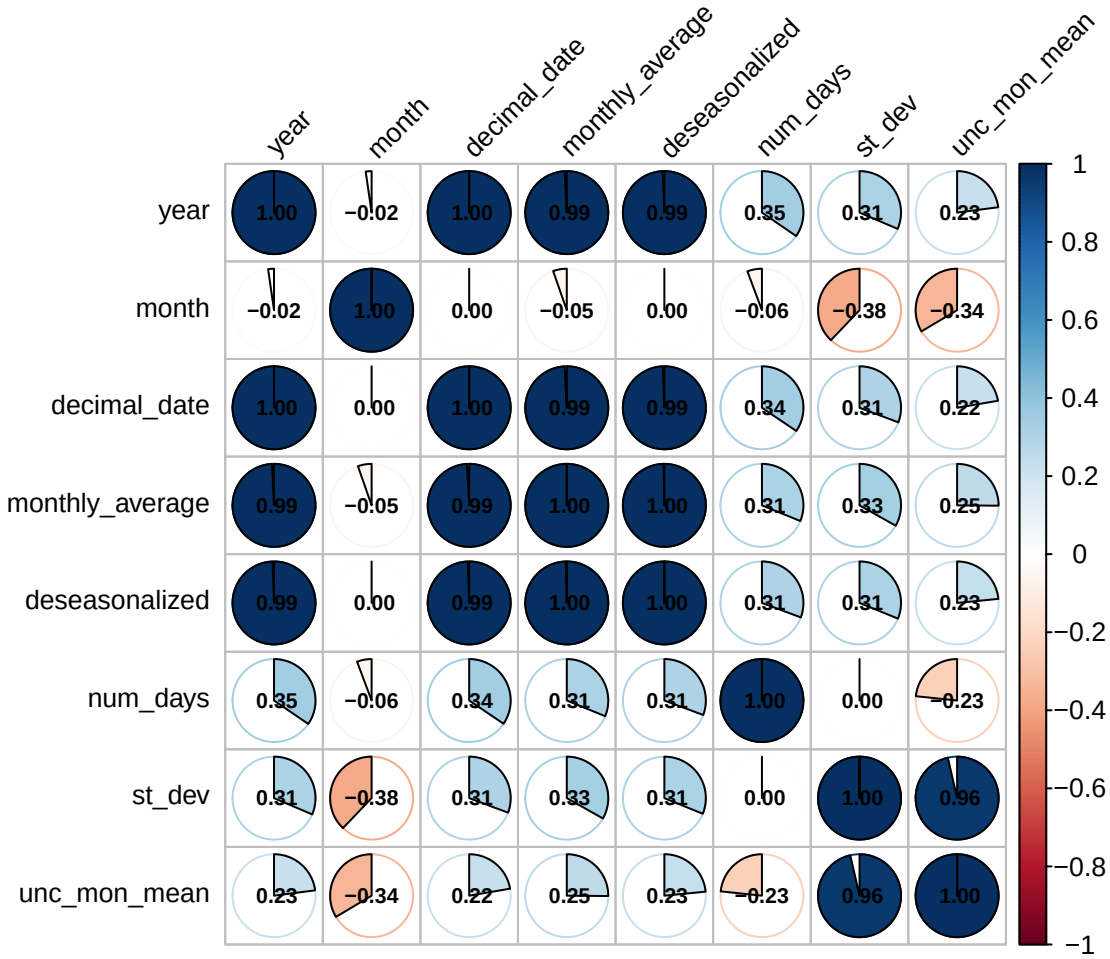

Figure 7: Correlation plot of the variables in the dataset

3. Bayesian Methodology and Models

See code 10 in Appendix for the implementation.

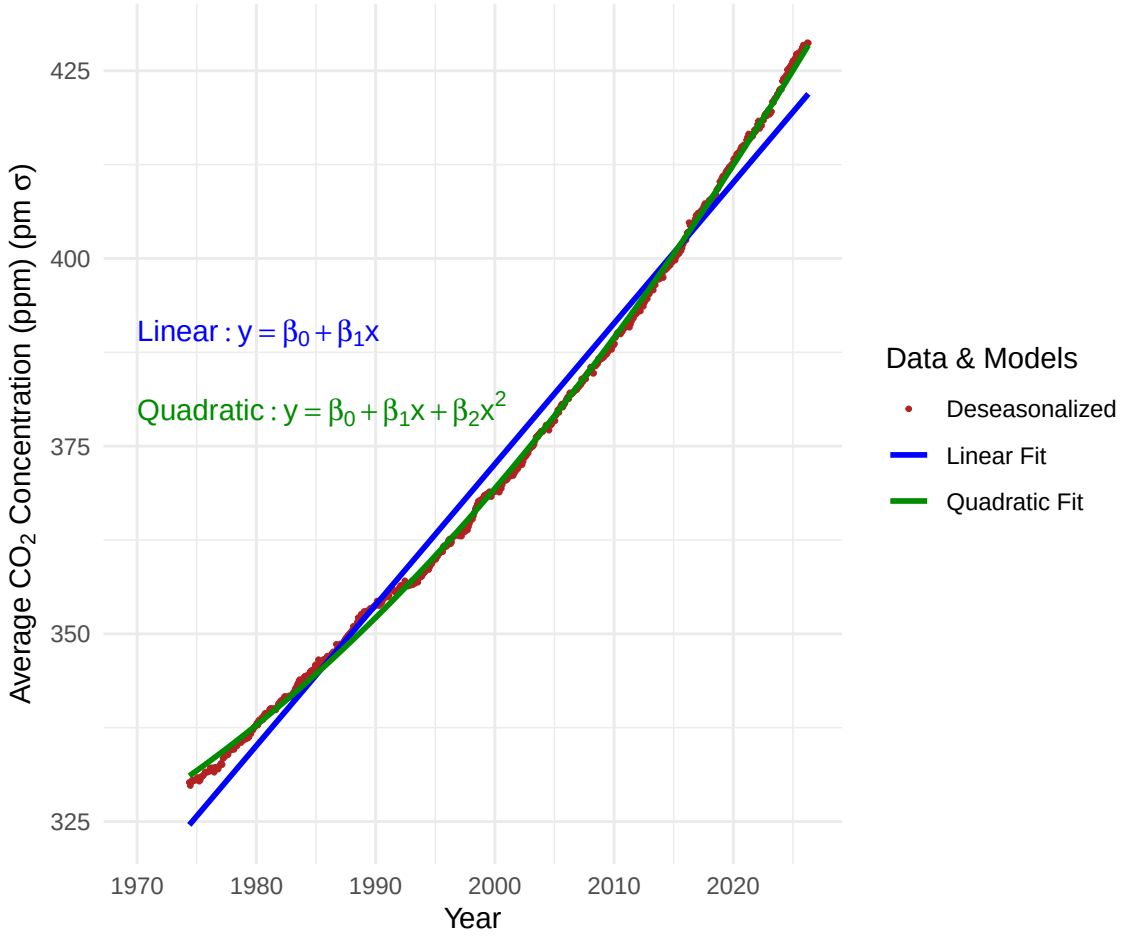

Figure 8: Deseasonalized CO2 concentrations and Bayesian models fit. The plot shows the deseasonalized CO2 concentrations along with the fitted lines from both the linear and quadratic models, illustrating the differences in their respective fits to the data.

In Figure 8, we can see the deseasonalized CO2 concentrations along with the fitted lines from both the linear and quadratic models, a priori, the quadratic model seems to capture the upward curvature in the data better than the linear model.

3.1. Model 1: Linear Trend

3.2. Model 2: Quadratic Trend

3.3. Inference Strategy

4. Results and Inference

4.1. Parameter Estimation

4.2. Model Comparison

5. Predictive Forecasting

5.1. Ten-Year Projections

5.2. Uncertainty Quantification

6. Conclusion

6.1. Summary of Findings

6.2. Limitations and Future Work

6.3. Appendix: Source code

```

source("code/Data.R")
library(skimr)
library(dplyr)
library(knitr)

# 1. Select the core variables you want to summarize
summary_data <- data %>%
  select(year, monthly_average, deseasonalized, num_days, st_dev, unc_mon_mean)

# 2. Skim the data, convert it to a regular data frame, and select clean
    summary metrics
skim_table <- skim(summary_data) %>%
  as.data.frame() %>%
  select(
    Variable = skim_variable,
    Missing = n_missing,
    Mean = numeric.mean,
    SD = numeric.sd,
    Min = numeric.p0,
    Median = numeric.p50,
    Max = numeric.p100
  )

kable(skim_table,
      digits = 2,
      booktabs = TRUE) # Makes the table look clean and academic
```

Listing 1: Plotting the data

```

source("code/Data.R")
library(skimr)
library(dplyr)
library(knitr)

# 1. Select the core variables you want to summarize
summary_data <- data_no_nan %>%
  select(year, monthly_average, deseasonalized, num_days, st_dev, unc_mon_mean)

# 2. Skim the data, convert it to a regular data frame, and select clean
    summary metrics
skim_table <- skim(summary_data) %>%
  as.data.frame() %>%
  select(
    Variable = skim_variable,
    Missing = n_missing,
    Mean = numeric.mean,
    SD = numeric.sd,
    Min = numeric.p0,
    Median = numeric.p50,
    Max = numeric.p100
  )

kable(skim_table,
      digits = 2,
      booktabs = TRUE) # Makes the table look clean and academic
```

Listing 2: Plotting the data

```

source("code/Data.R")
library(GGally)
library(dplyr)
library(ggplot2)

# 1. Clean the data subset and make 'month' a factor
eda_subset <- data_no_nan %>%
  select(monthly_average, deseasonalized, st_dev, unc_mon_mean, month, num_days
      ) %>%
  mutate(month = factor(month, labels = c("Jan", "Feb", "Mar", "Apr", "May", "
      Jun",
                                          "Jul", "Aug", "Sep", "Oct", "Nov", "
                                              Dec")))

# 2. Build explicit formatting wrappers for complete structural control
custom_scatter <- function(data, mapping, ...) {
  ggplot(data = data, mapping = mapping) +
    geom_point(alpha = 0.3, size = 0.3, color = "dimgray", ...)
}
```

```

custom_boxplot <- function(data, mapping, ...) {
  ggplot(data = data, mapping = mapping) +
    geom_boxplot(color = "darkcyan", fill = "lightcyan", outlier.size = 0.5,
        ...)
}

# NEW: Custom function to generate violin plots for data combinations
custom_violin <- function(data, mapping, ...) {
  ggplot(data = data, mapping = mapping) +
    geom_violin(color = "purple4", fill = "plum1", alpha = 0.5, scale = "width"
        , ...)
}

# 3. Generate the matrix with violin plots on top
ggpairs(
  eda_subset,
  title = "Distribution␣Matrix␣of␣CO2␣Variables",

  # Diagonal: Pure density curves for continuous data
  diag = list(
    continuous = wrap("densityDiag", fill = "lightsteelblue", alpha = 0.7),
    discrete = wrap("barDiag", fill = "lightsteelblue")
  ),

  # Lower Triangle: Scatter plots and box plots
  lower = list(
    continuous = custom_scatter
  ),

  # Upper Triangle: Violin plots for combos, density contours for continuous
      pairs
  upper = list(
    continuous = wrap("density", color = "darkslateblue", linewidth = 0.4),
    combo = custom_violin
  )
) +
  theme_minimal() +
  theme(
    panel.grid.major = element_blank(),
    axis.text = element_text(size = 7),
    strip.text = element_text(size = 9, face = "bold")
  )
```

Listing 3: Plotting the data

```

source("code/Data.R")
library(ggplot2)


```

```
ggplot(data_no_nan, aes(x = decimal_date)) +
  # 1. Error bars for monthly average
  geom_errorbar(aes(ymin = monthly_average - unc_mon_mean,
                    ymax = monthly_average + unc_mon_mean),
                width = 0.2,
                color = "darkgray",
                linewidth = 0.8) +

  # 2. Monthly Average Points
  geom_point(aes(y = monthly_average, color = "Monthly␣Average"), size = 0.5) +

  # 3. Deseasonalized Line & Points (Grouped under "Deseasonalized")
  geom_line(aes(y = deseasonalized, color = "Deseasonalized"), linewidth = 0.5)
      +
  geom_point(aes(y = deseasonalized, color = "Deseasonalized"), size = 0.8,
      alpha = 0.6) +

  scale_color_manual(
    name = "Data",
    values = c(
      "Monthly␣Average" = "black",
      "Deseasonalized"  = "firebrick"
    )
  ) +

  # Clean up the styling
  theme_minimal() +
  labs(
    x = "Year",
    y = expression(paste("Average␣CO" [2], "␣Concentration␣(ppm)␣(", pm, "␣",
        sigma, ")"))
  )
```

Listing 4: Plotting the data

```

source("code/Data.R")
library(ggplot2)

ggplot(data_no_nan, aes(x = year, y = monthly_average)) +
  # 1. Automatically calculate and plot the annual mean as bars
  stat_summary(fun = mean, geom = "bar", fill = "darkcyan", width = 0.8) +

  # 2. Automatically calculate and plot the standard deviation error bars
  # This fixes the ymin/ymax error by computing them on the fly from 'y'
  stat_summary(
    fun.min = function(x) mean(x) - sd(x),
    fun.max = function(x) mean(x) + sd(x),
    geom = "errorbar",
    width = 0.4,
    color = "darkgray",
```

```
    linewidth = 0.8
  ) +

  # Zoom into the y-axis cleanly so differences are visible
  coord_cartesian(ylim = c(300, 430)) +

  theme_minimal() +
  labs(
    x = "Year",
    y = expression(paste("Yearly␣Average␣CO" [2], "␣Concentration␣(ppm)␣(", pm,
        "␣", sigma, ")"))
  )
```

Listing 5: Plotting the data

```

source("code/Data.R")
library(ggplot2)

ggplot(data_no_nan, aes(x = num_days, y = monthly_average)) +
  # 1. Automatically calculate and plot the annual mean as bars
  stat_summary(fun = mean, geom = "bar", fill = "darkcyan", width = 0.8) +

  # 2. Automatically calculate and plot the standard deviation error bars
  # This fixes the ymin/ymax error by computing them on the fly from 'y'
  stat_summary(
    fun.min = function(x) mean(x) - sd(x),
    fun.max = function(x) mean(x) + sd(x),
    geom = "errorbar",
    width = 0.4,
    color = "darkgray",
    linewidth = 0.8
  ) +

  # Zoom into the y-axis cleanly so differences are visible
  #coord_cartesian(ylim = c(300, 430)) +

  theme_minimal() +
  labs(
    x = "Number␣of␣days␣measured␣in␣a␣month",
    y = expression(paste("Average␣CO" [2], "␣Concentration␣(ppm)␣(", pm, "␣",
        sigma, ")"))
  )
```

Listing 6: Plotting the data

```

source("code/Data.R")
library(ggplot2)

```

```r
data_no_nan$seasonal_deviation <- data_no_nan$monthly_average - data_no_nan$
    deseasonalized

ggplot(data_no_nan, aes(x = factor(month), y = seasonal_deviation)) +
  # Create a boxplot for each month
  geom_boxplot(fill = "lightblue", color = "darkblue", outlier.size = 0.5) +

  # Connect the monthly medians with a line to emphasize the cyclical wave
      shape
  stat_summary(aes(group = 1), fun = median, geom = "line", color = "firebrick"
      , linewidth = 1) +

  theme_minimal() +
  scale_x_discrete(labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                              "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")) +
  labs(
    title = "Climatological␣CO2␣Seasonal␣Fingerprint",
    x = "Month",
    y = expression(paste("Seasonal␣CO" [2], "␣Deviation␣(ppm)"))
  )
```

Listing 7: Plotting the data

```r

source("code/Data.R")
library(ggplot2)
library(patchwork)
library(dplyr)

# --- PLOT 1: Uncertainty and Std Dev Over Time ---
# We use geom_line to see if instrument precision or environmental variance
# has changed over the decades.
plot_time <- ggplot(data_no_nan, aes(x = decimal_date)) +
  # Plot standard deviation of daily measurements
  geom_line(aes(y = st_dev, color = "Daily␣St.␣Dev."), linewidth = 0.5, alpha =
       0.7) +
  # Plot calculated uncertainty of the monthly mean
  geom_line(aes(y = unc_mon_mean, color = "Monthly␣Mean␣Uncertainty"),
      linewidth = 0.5, alpha = 0.7) +

  scale_color_manual(
    name = "Metric",
    values = c("Daily␣St.␣Dev." = "dimgray", "Monthly␣Mean␣Uncertainty" = "
        firebrick")
  ) +
  theme_minimal() +
  theme(legend.position = "bottom") +
  labs(
    x = "Year",
    y = "Uncertainty␣/␣Standard␣Deviation␣(ppm)",
    title = "Uncertainty␣Trends␣Over␣Time"
```

```
  )

# --- PLOT 2: Standard Deviation by Month ---
# This checks if the variation in daily CO2 changes depending on the season
# (e.g., higher variance during summer drawdown months).
plot_monthly <- ggplot(data_no_nan, aes(x = factor(month), y = st_dev)) +
  # Boxplot to show the distribution of standard deviations for each month
      across all years
  geom_boxplot(fill = "lightsteelblue", color = "black", outlier.size = 0.5) +
  # Add a trend line connecting the monthly means
  stat_summary(aes(group = 1), fun = mean, geom = "line", color = "darkblue",
      linewidth = 1) +

  scale_x_discrete(labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                              "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")) +
  theme_minimal() +
  labs(
    x = "Month",
    y = "Daily Standard Deviation (ppm)",
    title = "Daily Variance by Season"
  )

# --- COMBINE THEM SIDE-BY-SIDE ---
combined_uncertainty <- plot_time + plot_monthly +
  plot_annotation(
    title = "Data Quality and Heteroskedasticity Assessment",
    tag_levels = "A"
  )

# Display the final layout
combined_uncertainty
```

Listing 8: Plotting the data

```

# This is an example of a test file for EDA (Exploratory Data Analysis)
source("code/Data.R")
library(corrplot)

corrplot(cor(data_no_nan), method = "pie", addCoef.col = "black", tl.cex = 0.8,
     number.cex = 0.7,
         tl.col = "black", tl.srt = 45, addrect = 2, rect.col = "blue", rect.
             lwd = 1.5)
```

Listing 9: Correlating the data

```

source("code/Data.R")
library(ggplot2)

```

```
ggplot(data_no_nan, aes(x = decimal_date)) +
  # 1. Error bars for monthly average

  # 2. Monthly Average Points
  geom_point(aes(y = deseasonalized, color = "Deseasonalized"), size = 0.5) +


  # 4. Trend lines mapped to custom colors in aes()
  geom_smooth(aes(y = deseasonalized, color = "Linear Fit"), method = "lm",
      formula = y ~ x, se = FALSE) +
  geom_smooth(aes(y = deseasonalized, color = "Quadratic Fit"), method = "lm",
      formula = y ~ x + I(x^2), se = FALSE) +

  # 5. EQUATION LABELS: Manually placed in empty areas of the graph
  # Adjust the 'x' and 'y' coordinates to fit your data's exact range perfectly
  annotate("text", x = 1970, y = 390, label = "Linear: y == beta[0] + beta[1]*x
      ",
           color = "blue", parse = TRUE, hjust = 0, size = 4) +
  annotate("text", x = 1970, y = 380, label = "Quadratic: y == beta[0] + beta
      [1]*x + beta[2]*x^2",
           color = "green4", parse = TRUE, hjust = 0, size = 4) +

  # 6. Explicitly define your custom color scheme for the legend
  scale_color_manual(
    name = "Data & Models",
    values = c(
      "Deseasonalized"  = "firebrick",
      "Linear Fit"      = "blue",
      "Quadratic Fit"   = "green4" # A slightly darker green for better
          visibility
    )
  ) +

  # Clean up the styling
  theme_minimal() +
  labs(
    x = "Year",
    y = expression(paste("Average CO" [2], " Concentration (ppm) (", pm, " ",
        sigma, ")"))
  )
```

Listing 10: Plotting the data

References